

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 12. Sets

Lecture: 127. Internal Working of Sets

Section 1: Lecture Summary

This lecture explains how Python sets are represented internally using a hash table and the hashing technique. Unlike lists and tuples, which are simple arrays with sequential indices and maintain insertion order, sets use hashing for element storage, which means elements are placed and accessed based on hash function calculations rather than insertion order. The lecture clarifies hashing by using a mod function example, illustrating element placement, handling collisions with chaining, and the impact of hash table size. It also explores how adding elements can cause the hash table to resize, changing the internal order of elements. The key advantage of sets is that insertion and search operations have constant time complexity ($O(1)$) due to hashing.

Section 2: Key Concepts and Explanations

- Lists and Tuples Internal Structure

- Both are array-like structures storing elements at sequential indices (0, 1, 2, ...).
- Lists are mutable (can be modified), tuples are immutable (read-only).

- Set Internal Representation

- Sets are also stored internally as arrays called hash tables.
- Elements in sets are referred to as keys.

- Hashing and Hash Functions

- A hash function maps a key to an index in the hash table.

- Example hash function demonstrated: $h(x) = x \bmod m$, where m is the hash table size.

- Example with size 10: keys like 5, 10, 21 are stored at indices based on their mod 10 result.

- Collisions occur when two keys map to the same index (e.g., 5 and 15 both map to index 5).

- Collisions are resolved using chaining (linked lists at each index).

- Non-Sequential Ordering in Sets

- Since elements are stored according to hash indices, insertion order is not preserved.

- Sets do not support index-based access like lists or tuples.

- Complexity of Operations

- Both insertion and search operations in a set take average constant time $O(1)$.

- This is because hash function computations and accessing the hash table slot take constant time.

- Inefficient hash functions causing many collisions degrade performance but are considered a design flaw of the hash function, not the hash table mechanism.

- Hash Table Resizing

- When the hash table reaches a certain load factor (e.g., 50% or 75%), it resizes (often grows larger, sometimes by a factor of 4).

- After resizing, the hash function updates accordingly, changing how keys map to indices.

- This resizing alters the internal order of elements in the set, explaining why set element order can change unexpectedly after adding more elements.

- Python Set Internal Guesswork

- The lecture guesses Python uses an initial hash table size of 16 and a hash function similar to $x \bmod 16$.

- Adding elements beyond half capacity causes resize and rehashing, changing element order.
- This guess matches observed behavior, though Python's actual implementation may differ internally.

Section 3: Example Code and Use Cases

Create a set with initial elements

```
s = set()
elements = [10, 20, 30, 40, 50]
for el in elements:
    s.add(el)

print(s) # Output might be: {50, 20, 40, 10, 30}
```

This example demonstrates adding elements to a set and printing it to observe the unordered storage.

Add more elements to observe order change upon hash table resize

```
s.add(60) # 60 mod 16 = 12
s.add(70) # 70 mod 16 = 6
print(s) # Output order changes, e.g. {50, 20, 70, 40, 10, 60, 30}
```

Adding elements 60 and 70 changes the internal structure and order.

```
s.add(18) # 18 mod 16 = 2, lands near 50 which is at index 2
print(s) # Shows 18 inserted just after 50
```

```
s.add(31) # 31 mod 16 = 15, possibly triggers resize; order changes again
print(s)
```

Inserting 31 causes the hash table to resize and elements reorder.

Section 4: Key Takeaways

- Sets internally use hash tables for storing elements, not simple arrays like lists or tuples.
- Elements do not have fixed indices and insertion order is not maintained.
- A hash function determines where elements are stored; collisions are handled by chaining.
- Insertion and lookup in sets have average constant time complexity $O(1)$.
- The efficiency of sets depends heavily on the quality of the hash function to distribute keys uniformly.
- When the hash table reaches a threshold load, it resizes and rehashes elements, which alters their order.
- Python's actual internal hash function and resizing policy are abstracted, but behavior aligns with this hashing model.
- Understanding hash tables is fundamental to appreciating why sets are faster than lists and tuples for membership testing and insertions.

